Claude Certified Architect – Foundations

Guida all'esame

Versione 1.0 · In vigore da luglio 2026 · Codice esame: CCAR-F

La presente guida è soggetta a modifiche senza preavviso.

1. Informazioni sulla certificazione

La certificazione Claude Certified Architect – Foundations attesta che i professionisti sono in grado di valutare consapevolmente i compromessi da considerare quando implementano soluzioni concrete con Claude. L'esame verifica le conoscenze fondamentali di Claude Code, Claude Agent SDK, Claude API e Model Context Protocol (MCP), le tecnologie principali per sviluppare applicazioni di livello produttivo basate su Claude.

Le domande si basano su scenari realistici tratti da casi d'uso effettivi dei clienti: creazione di sistemi agentici per l'assistenza clienti, progettazione di pipeline di ricerca multi-agente, integrazione di Claude Code nei flussi CI/CD, sviluppo di strumenti per la produttività degli sviluppatori ed estrazione di dati strutturati da documenti non strutturati. I candidati devono dimostrare non solo conoscenze concettuali, ma anche capacità di valutare architettura, configurazione e compromessi nelle distribuzioni di produzione.

Questa guida è il riferimento ufficiale per chi si prepara all'esame. Descrive i contenuti, elenca gli ambiti e gli obiettivi valutati, propone domande di esempio e suggerisce strategie di preparazione. Leggila per intero prima di prenotare l'esame.

2. Destinatari

Il candidato ideale è un architetto di soluzioni che progetta e implementa applicazioni di produzione con Claude e ha esperienza pratica in:

- sviluppo di applicazioni agentiche con Claude Agent SDK, inclusi orchestrazione multi-agente, delega ai sottoagenti, integrazione di strumenti e hook del ciclo di vita;
- configurazione e personalizzazione di Claude Code per i flussi di lavoro dei team tramite file CLAUDE.md, Agent Skills, integrazioni con server MCP e modalità Plan;
- progettazione di interfacce di strumenti e risorse MCP per l'integrazione con sistemi di backend;
- progettazione di prompt che producono output strutturati affidabili, usando schemi JSON, esempi few-shot e tecniche di estrazione;
- gestione efficace delle finestre di contesto in documenti lunghi, conversazioni multi-turno e passaggi tra sottoagenti;
- integrazione di Claude nelle pipeline CI/CD per revisioni automatiche del codice, generazione di test e feedback sulle pull request;
- decisioni affidabili su escalation e robustezza, inclusi gestione degli errori, flussi con intervento umano e modelli di autovalutazione.

In genere, il candidato ha almeno sei mesi di esperienza pratica nello sviluppo con Claude API, Agent

SDK, Claude Code e MCP e conosce sia le capacità sia i limiti dei modelli linguistici di grandi dimensioni negli ambienti di produzione.

3. Dettagli dell'esame in breve

Credenziale	Claude Certified Architect – Foundations
Codice esame	CCAR-F
Numero di domande	60
Formato	Domande a scelta singola e multipla; ogni domanda indica quante risposte selezionare
Struttura	4 scenari estratti da un insieme di 6
Durata	120 minuti
Modalità	Esame sorvegliato online e/o presso un centro d'esame, secondo le regole del programma
Punteggio minimo	Punteggio ponderato di 720 su una scala da 100 a 1.000
Costo	125 USD
Validità	12 mesi dalla data di assegnazione della credenziale
Risultati	Esito superato/non superato con punteggio ponderato (100–1.000) e percentuale di risposte corrette per dominio nel rapporto dei risultati

4. Struttura dei contenuti dell'esame (blueprint)

Il blueprint definisce gli ambiti di contenuto valutati e il peso approssimativo di ciascuno. I pesi riflettono l'importanza relativa di ogni ambito per una prestazione competente, così come determinata dall'analisi delle attività professionali. Le percentuali indicano la quota approssimativa di domande valutate assegnata a ciascun ambito.

1	**Architettura agentica e orchestrazione**	**27%**
2	Progettazione degli strumenti e integrazione MCP	18%
3	Configurazione e flussi di lavoro di Claude Code	20%
4	Progettazione dei prompt e output strutturato	20%
5	Gestione del contesto e affidabilità	15%
	Totale	100%

5. Scenari d'esame

L'esame presenta domande basate su scenari. Ogni scenario descrive un contesto realistico di produzione che fa da cornice a una serie di domande. Durante l'esame ne vengono presentati 4, scelti casualmente dai 6 seguenti.

Scenario 1: agente per la risoluzione delle richieste di assistenza clienti

Stai sviluppando con Claude Agent SDK un agente che gestisce richieste ambigue, come resi, contestazioni di addebiti e problemi relativi agli account. L'agente accede ai sistemi di backend tramite strumenti MCP personalizzati (get_customer, lookup_order, process_refund, escalate_to_human). L'obiettivo è risolvere oltre l'80% delle richieste al primo contatto, riconoscendo quando occorre effettuare un'escalation.

Domini principali: Architettura agentica e orchestrazione; progettazione degli strumenti e integrazione MCP; gestione del contesto e affidabilità.

Scenario 2: generazione di codice con Claude Code

Usi Claude Code per accelerare lo sviluppo software. Il team lo impiega per generare codice, effettuare refactoring, eseguire debugging e redigere documentazione. Devi integrarlo nel flusso di sviluppo con comandi slash personalizzati e configurazioni CLAUDE.md, e stabilire quando usare la modalità Plan o l'esecuzione diretta.

Domini principali: Configurazione e flussi di lavoro di Claude Code; gestione del contesto e affidabilità.

Scenario 3: sistema di ricerca multi-agente

Stai sviluppando con Claude Agent SDK un sistema di ricerca multi-agente. Un agente coordinatore delega il lavoro a sottoagenti specializzati: uno cerca sul web, uno analizza documenti, uno sintetizza i risultati e uno genera rapporti. Il sistema svolge ricerche su diversi argomenti e produce rapporti completi con citazioni.

Domini principali: Architettura agentica e orchestrazione; progettazione degli strumenti e integrazione MCP; gestione del contesto e affidabilità.

Scenario 4: produttività degli sviluppatori con Claude

Stai creando strumenti per la produttività degli sviluppatori con Claude Agent SDK. L'agente aiuta gli ingegneri a esplorare basi di codice sconosciute, comprendere sistemi legacy, generare codice standard e automatizzare attività ripetitive. Usa gli strumenti integrati (Read, Write, Bash, Grep, Glob) e si integra con server MCP.

Domini principali: Progettazione degli strumenti e integrazione MCP; configurazione e flussi di lavoro di Claude Code; architettura agentica e orchestrazione.

Scenario 5: Claude Code per l'integrazione continua

Stai integrando Claude Code nella pipeline di integrazione e distribuzione continua (CI/CD). Il sistema esegue revisioni automatiche del codice, genera casi di test e fornisce feedback sulle pull request. Devi progettare prompt che producano osservazioni utili e riducano al minimo i falsi positivi.

Domini principali: Configurazione e flussi di lavoro di Claude Code; progettazione dei prompt e output strutturato.

Scenario 6: estrazione di dati strutturati

Stai sviluppando con Claude un sistema per estrarre dati strutturati. Il sistema estrae informazioni da documenti non strutturati, convalida l'output tramite schemi JSON e mantiene un'elevata accuratezza. Deve gestire correttamente i casi limite e integrarsi con i sistemi a valle.

Domini principali: Progettazione dei prompt e output strutturato; gestione del contesto e affidabilità.

6. Obiettivi dettagliati per dominio

Ogni dominio elenca gli obiettivi d'esame e le conoscenze e competenze valutate. Le domande si basano su questi obiettivi.

Dominio 1: architettura agentica e orchestrazione

Obiettivo 1.1: progettare e implementare cicli agentici per l'esecuzione autonoma di attività

Conoscenze:

- ciclo di vita del loop agentico: inviare richieste a Claude, esaminare stop_reason (tool_use o end_turn), eseguire gli strumenti richiesti e restituirne i risultati per l'iterazione successiva;
- modalità con cui i risultati degli strumenti vengono aggiunti alla cronologia della conversazione, affinché il modello possa ragionare sull'azione successiva;
- differenza tra decisioni guidate dal modello (Claude sceglie il prossimo strumento in base al contesto) e sequenze di strumenti o alberi decisionali preconfigurati.

Competenze:

- implementare il flusso di controllo del loop agentico affinché continui quando stop_reason è tool_use e termini quando è end_turn;
- aggiungere i risultati degli strumenti al contesto della conversazione tra un'iterazione e l'altra, così che il modello possa integrarli nel ragionamento;
- evitare pratiche scorrette come interpretare segnali in linguaggio naturale per terminare il loop, usare limiti arbitrari di iterazioni come criterio principale di arresto o considerare il testo dell'assistente un indicatore di completamento.

Obiettivo 1.2: orchestrare sistemi multi-agente con un coordinatore e sottoagenti

Conoscenze:

- architettura hub-and-spoke, in cui un agente coordinatore gestisce le comunicazioni tra sottoagenti, la gestione degli errori e l'instradamento delle informazioni;
- contesto isolato dei sottoagenti: non ereditano automaticamente la cronologia della conversazione del coordinatore;
- ruolo del coordinatore nella scomposizione delle attività, nella delega, nell'aggregazione dei risultati e nella scelta dei sottoagenti da attivare in base alla complessità della richiesta;
- rischi di una scomposizione eccessivamente circoscritta da parte del coordinatore, che può lasciare incompleta la copertura di temi di ricerca ampi.

Competenze:

- progettare coordinatori che analizzino le esigenze della richiesta e selezionino dinamicamente i sottoagenti da attivare, invece di instradare sempre il lavoro attraverso l'intera pipeline;
- suddividere l'ambito della ricerca tra sottoagenti per ridurre le duplicazioni, assegnando per esempio argomenti secondari o tipi di fonti distinti;
- implementare cicli iterativi di perfezionamento in cui il coordinatore individua lacune nella sintesi, delega nuovamente ricerche e analisi tramite richieste mirate e ripete la sintesi finché la copertura non è sufficiente;

- far passare tutte le comunicazioni dei sottoagenti dal coordinatore, per garantire osservabilità, gestione coerente degli errori e controllo del flusso informativo.

Obiettivo 1.3: configurare l'invocazione dei sottoagenti, il passaggio del contesto e la loro creazione

Conoscenze:

- lo strumento Task consente di creare sottoagenti; allowedTools deve includere Task affinché il coordinatore possa invocarli;
- il contesto dei sottoagenti deve essere fornito esplicitamente nel prompt: non ereditano automaticamente il contesto del genitore e non condividono la memoria tra invocazioni;
- configurazione di AgentDefinition, incluse descrizioni, prompt di sistema e restrizioni sugli strumenti per ogni tipo di sottoagente;
- gestione delle sessioni basata su fork per esplorare approcci diversi a partire da una base di analisi condivisa.

Competenze:

- includere direttamente nel prompt del sottoagente i risultati completi degli agenti precedenti, per esempio i risultati della ricerca web e dell'analisi documentale da passare all'agente di sintesi;
- usare formati di dati strutturati per separare il contenuto dai metadati (URL delle fonti, nomi dei documenti, numeri di pagina) quando si passa il contesto, in modo da preservare l'attribuzione;
- avviare sottoagenti in parallelo emettendo più chiamate allo strumento Task in un'unica risposta del coordinatore, invece che in turni separati;
- progettare prompt per il coordinatore che definiscano obiettivi di ricerca e criteri di qualità, anziché istruzioni procedurali passo per passo, così da consentire ai sottoagenti di adattarsi.

Obiettivo 1.4: implementare flussi di lavoro multi-fase con vincoli e passaggio di consegne

Conoscenze:

- differenza tra applicazione programmatica dei vincoli (hook e prerequisiti) e indicazioni nei prompt sull'ordine del flusso di lavoro;
- nei casi in cui è necessaria conformità deterministica (per esempio verificare l'identità prima di operazioni finanziarie), le sole istruzioni nel prompt hanno una probabilità di errore diversa da zero;
- protocolli strutturati per l'escalation durante un processo, con dettagli del cliente, analisi della causa principale e azioni consigliate.

Competenze:

- implementare prerequisiti programmatici che blocchino le chiamate agli strumenti successivi finché i passaggi necessari non sono completati, per esempio impedire process_refund finché get_customer non restituisce un ID cliente verificato;
- suddividere le richieste dei clienti che comprendono più problemi in elementi distinti, esaminandoli in parallelo con un contesto condiviso prima di formulare una risposta unificata;
- preparare riepiloghi strutturati per il passaggio a operatori umani (ID cliente, causa principale, importo del rimborso, azione consigliata) che non hanno accesso alla trascrizione della conversazione.

Obiettivo 1.5: applicare gli hook di Agent SDK per intercettare le chiamate agli strumenti e normalizzare i dati

Conoscenze:

- pattern di hook, come PostToolUse, che intercettano i risultati degli strumenti per trasformarli prima che il modello li elabori;
- pattern di hook che intercettano le chiamate agli strumenti in uscita per applicare regole di conformità, per esempio bloccare rimborsi sopra una certa soglia;
- differenza tra l'uso degli hook per garanzie deterministiche e l'affidamento alle istruzioni nei prompt per una conformità probabilistica.

Competenze:

- implementare hook PostToolUse per normalizzare formati di dati eterogenei (timestamp Unix, ISO 8601, codici di stato numerici) restituiti da diversi strumenti MCP prima che l'agente li elabori;
- implementare hook che intercettino le chiamate agli strumenti, blocchino azioni contrarie alle regole (per esempio rimborsi superiori a 500 USD) e indirizzino verso flussi alternativi, come l'escalation a un operatore;
- scegliere gli hook al posto dell'applicazione dei vincoli tramite prompt quando le regole aziendali richiedono conformità garantita.

Obiettivo 1.6: progettare strategie di scomposizione delle attività per flussi di lavoro complessi

Conoscenze:

- quando usare pipeline sequenziali fisse (prompt chaining) e quando una scomposizione dinamica e adattiva basata sui risultati intermedi;
- pattern di prompt chaining che suddividono una revisione in passaggi sequenziali, per esempio analizzare ogni file singolarmente e poi eseguire una verifica d'integrazione tra i file;
- valore dei piani d'indagine adattivi, che generano sottoattività in base a ciò che emerge a ogni passaggio.

Competenze:

- scegliere il modello di scomposizione adatto al flusso di lavoro: prompt chaining per revisioni prevedibili e articolate in più aspetti, scomposizione dinamica per indagini aperte;
- suddividere le revisioni di codice ampie in analisi locali per singolo file e in un passaggio d'integrazione separato tra file, per evitare la dispersione dell'attenzione;
- scomporre attività aperte, come «aggiungere test completi a una base di codice legacy», mappando prima la struttura, individuando le aree a maggiore impatto e creando poi un piano prioritario che si adatti alle dipendenze scoperte.

Obiettivo 1.7: gestire lo stato delle sessioni, la ripresa e i fork

Conoscenze:

- ripresa di una sessione nominata con --resume <session-name> per continuare una conversazione precedente specifica;
- fork_session crea rami indipendenti a partire da una base di analisi condivisa, per esplorare approcci diversi;
- quando si riprende una sessione dopo aver modificato il codice, è importante informare l'agente dei cambiamenti nei file già analizzati;
- perché, in alcuni casi, avviare una nuova sessione con un riepilogo strutturato è più affidabile che riprendere con risultati degli strumenti ormai obsoleti.

Competenze:

- usare --resume con i nomi delle sessioni per continuare indagini nominate tra diverse sessioni di lavoro;
- usare fork_session per creare esplorazioni parallele, per esempio confrontando strategie di test o approcci di refactoring diversi a partire dall'analisi della stessa base di codice;
- scegliere tra riprendere una sessione, quando la maggior parte del contesto precedente è ancora valida, e ripartire da zero con riepiloghi inseriti nel contesto, quando i risultati precedenti degli strumenti sono obsoleti;
- segnalare a una sessione ripresa le modifiche specifiche ai file per consentire una nuova analisi mirata, senza dover ripetere l'intera esplorazione.

Dominio 2: progettazione degli strumenti e integrazione MCP

Obiettivo 2.1: progettare interfacce efficaci per gli strumenti, con descrizioni e limiti chiari

Conoscenze:

- le descrizioni degli strumenti sono il meccanismo principale con cui i modelli linguistici di grandi dimensioni scelgono gli strumenti; descrizioni troppo brevi rendono inaffidabile la scelta tra strumenti simili;
- è importante includere nelle descrizioni formati di input, richieste di esempio, casi limite e spiegazioni dei confini d'uso;
- descrizioni ambigue o sovrapposte possono instradare le richieste allo strumento sbagliato, per esempio analyze_content e analyze_document con descrizioni quasi identiche;
- le parole usate nel prompt di sistema possono influenzare la scelta degli strumenti: istruzioni sensibili alle parole chiave possono creare associazioni indesiderate.

Competenze:

- scrivere descrizioni che distinguano chiaramente scopo, input e output previsti di ogni strumento e spieghino quando usarlo invece di alternative simili;
- rinominare gli strumenti e aggiornarne le descrizioni per eliminare sovrapposizioni funzionali, per esempio rinominare analyze_content in extract_web_results e specificare l'uso web;
- suddividere strumenti generici in strumenti specializzati con contratti di input e output definiti, per esempio sostituire analyze_document con extract_data_points, summarize_content e verify_claim_against_source;
- esaminare i prompt di sistema per individuare istruzioni sensibili alle parole chiave che potrebbero prevalere su descrizioni degli strumenti ben scritte.

Obiettivo 2.2: implementare risposte d'errore strutturate per gli strumenti MCP

Conoscenze:

- pattern del flag isError di MCP per comunicare gli errori degli strumenti all'agente;
- differenza tra errori temporanei (timeout, servizio non disponibile), errori di convalida (input non valido), errori aziendali (violazioni delle regole) ed errori di autorizzazione;
- risposte d'errore uniformi e generiche, come «Operazione non riuscita», impediscono all'agente di scegliere una strategia di recupero appropriata;
- differenza tra errori ripetibili e non ripetibili, e come i metadati strutturati evitino tentativi di ripetizione inutili.

Competenze:

- restituire metadati d'errore strutturati che includano errorCategory (transient, validation, permission), un

booleano isRetryable e una descrizione comprensibile;
- includere il flag retriable: false e spiegazioni comprensibili al cliente per le violazioni delle regole aziendali, affinché l'agente possa comunicarle correttamente;
- implementare il recupero locale dagli errori temporanei nei sottoagenti e segnalare al coordinatore solo gli errori non risolvibili localmente, insieme ai risultati parziali e alle operazioni già tentate;
- distinguere gli errori di accesso, che richiedono una decisione sui tentativi successivi, dai risultati vuoti validi, che indicano una ricerca riuscita senza corrispondenze.

Obiettivo 2.3: distribuire gli strumenti tra gli agenti e configurare la scelta degli strumenti

Conoscenze:

- fornire a un agente troppi strumenti, per esempio 18 invece di 4 o 5, riduce l'affidabilità della selezione aumentando la complessità della decisione;
- gli agenti tendono a usare in modo improprio gli strumenti che non rientrano nella loro specializzazione, per esempio un agente di sintesi che tenta ricerche sul web;
- accesso circoscritto agli strumenti: fornire a ogni agente solo quelli necessari al proprio ruolo, con un numero limitato di strumenti condivisi per esigenze frequenti;
- opzioni di configurazione di tool_choice: auto, any e selezione forzata ({"type":"tool","name":"..."}).

Competenze:

- limitare l'insieme di strumenti di ogni sottoagente a quelli pertinenti al suo ruolo, evitando usi impropri tra specializzazioni;
- sostituire strumenti generici con alternative circoscritte, per esempio sostituire fetch_url con load_document, che convalida gli URL dei documenti;
- fornire strumenti condivisi circoscritti per esigenze frequenti, per esempio uno strumento verify_fact per l'agente di sintesi, lasciando al coordinatore i casi complessi;
- usare la selezione forzata tool_choice per assicurarsi che venga chiamato per primo uno strumento specifico, per esempio extract_metadata, e poi elaborare i passaggi successivi in turni seguenti;
- impostare tool_choice: "any" per garantire che il modello chiami uno strumento invece di restituire testo conversazionale.

Obiettivo 2.4: integrare server MCP in Claude Code e nei flussi di lavoro degli agenti

Conoscenze:

- ambito dei server MCP: configurazione a livello di progetto (.mcp.json) per gli strumenti condivisi dal team e a livello utente (~/.claude.json) per server personali o sperimentali;
- espansione delle variabili d'ambiente in .mcp.json (per esempio ${GITHUB_TOKEN}) per gestire le credenziali senza salvare segreti nel repository;
- gli strumenti di tutti i server MCP configurati vengono individuati al momento della connessione e sono disponibili contemporaneamente all'agente;
- le risorse MCP espongono cataloghi di contenuti (per esempio riepiloghi di issue, gerarchie documentali, schemi di database) riducendo le chiamate esplorative agli strumenti.

Competenze:

- configurare server MCP condivisi nel file .mcp.json a livello di progetto, espandendo variabili d'ambiente per i token di autenticazione;
- configurare server MCP personali o sperimentali in ~/.claude.json;
-

migliorare le descrizioni degli strumenti MCP spiegandone in dettaglio capacità e output, così da evitare che l'agente preferisca strumenti integrati, come Grep, a strumenti MCP più adeguati;
- preferire server MCP esistenti e mantenuti dalla community per integrazioni standard, come Jira, e riservare i server personalizzati ai flussi di lavoro specifici del team;
- esporre cataloghi di contenuti come risorse MCP per rendere visibili i dati disponibili agli agenti senza richiedere chiamate esplorative agli strumenti.

Obiettivo 2.5: selezionare e usare efficacemente gli strumenti integrati (Read, Write, Edit, Bash, Grep, Glob)

Conoscenze:

- Grep cerca contenuti, per esempio nomi di funzioni, messaggi di errore o istruzioni di importazione;
- Glob individua i percorsi dei file in base a pattern, per esempio nome o estensione;
- Read e Write eseguono operazioni complete sui file; Edit applica modifiche mirate usando testo univoco come riferimento;
- quando Edit non riesce perché il testo corrisponde a più punti, usare Read e poi Write come alternativa affidabile.

Competenze:

- scegliere Grep per cercare contenuti nel codice di una base di codice, per esempio tutti i punti in cui viene chiamata una funzione o la presenza di messaggi di errore;
- scegliere Glob per individuare file che corrispondono a pattern, per esempio /*.test.tsx;
- usare Read per caricare l'intero contenuto di un file e poi Write quando Edit non trova un riferimento univoco;
- comprendere la base di codice per gradi: iniziare con Grep per trovare i punti d'ingresso, poi usare Read per seguire gli import e tracciare i flussi, invece di leggere subito tutti i file;
- tracciare l'uso delle funzioni attraverso moduli wrapper individuando prima tutti i nomi esportati e poi cercando ciascun nome nella base di codice.

Dominio 3: configurazione e flussi di lavoro di Claude Code

Obiettivo 3.1: configurare i file CLAUDE.md con gerarchia, ambito e struttura modulari appropriati

Conoscenze:

- gerarchia di configurazione di CLAUDE.md: livello utente (~/.claude/CLAUDE.md), livello progetto (.claude/CLAUDE.md o CLAUDE.md nella radice) e livello directory (file CLAUDE.md nelle sottodirectory);
- le impostazioni a livello utente si applicano solo a quell'utente: le istruzioni in ~/.claude/CLAUDE.md non sono condivise con i colleghi tramite il controllo versione;
- sintassi @import per fare riferimento a file esterni e mantenere modulare CLAUDE.md, per esempio importando gli standard pertinenti a ciascun pacchetto;
- directory .claude/rules/ per organizzare file di regole specifiche per argomento, in alternativa a un unico CLAUDE.md monolitico.

Competenze:

- diagnosticare problemi nella gerarchia di configurazione, per esempio quando un nuovo membro del team non riceve istruzioni perché sono salvate a livello utente e non di progetto;

- usare @import per includere selettivamente in ogni CLAUDE.md di un pacchetto gli standard pertinenti, in base alle conoscenze dei responsabili;
- suddividere file CLAUDE.md di grandi dimensioni in file tematici mirati nella directory .claude/rules/, come testing.md, api-conventions.md e deployment.md;
- usare il comando /memory per verificare quali file di memoria sono caricati e diagnosticare comportamenti incoerenti tra sessioni.

Obiettivo 3.2: creare e configurare comandi slash personalizzati e skill

Conoscenze:

- comandi a livello di progetto in .claude/commands/, condivisi tramite controllo versione, e comandi a livello utente in ~/.claude/commands/, destinati all'uso personale;
- skill in .claude/skills/ con file SKILL.md e configurazione frontmatter, che supporta context: fork, allowed-tools e argument-hint;
- opzione frontmatter context: fork, che esegue una skill in un contesto isolato di sottoagente, evitando che il suo output contamini la conversazione principale;
- personalizzazione delle skill: creare varianti personali in ~/.claude/skills/ con nomi diversi per non influire sui colleghi.

Competenze:

- creare comandi slash a livello di progetto in .claude/commands/, disponibili a tutto il team tramite controllo versione;
- usare context: fork per isolare dalla sessione principale le skill che producono molto output, per esempio nell'analisi della base di codice, oppure contenuti esplorativi, come il brainstorming di alternative;
- configurare allowed-tools nel frontmatter della skill per limitare gli strumenti accessibili durante l'esecuzione, per esempio alle sole operazioni di scrittura sui file per impedire azioni distruttive;
- usare il frontmatter argument-hint per chiedere agli sviluppatori i parametri necessari quando invocano una skill senza argomenti;
- scegliere tra le skill, invocate su richiesta per flussi specifici, e CLAUDE.md, che contiene standard universali sempre caricati.

Obiettivo 3.3: applicare regole specifiche per percorso per caricare convenzioni in modo condizionale

Conoscenze:

- file .claude/rules/ con campi paths nel frontmatter YAML, contenenti pattern glob per attivare le regole in modo condizionale;
- le regole circoscritte a un percorso vengono caricate solo quando si modificano file corrispondenti, riducendo il contesto irrilevante e l'uso di token;
- i pattern glob nelle regole sono preferibili ai file CLAUDE.md nelle singole directory quando le convenzioni riguardano più directory, per esempio file di test distribuiti nella base di codice.

Competenze:

- creare file .claude/rules/ con ambiti specificati nel frontmatter YAML, per esempio paths: ["terraform//*"], affinché le regole vengano caricate solo quando si modificano file corrispondenti;
- usare pattern glob nelle regole specifiche per percorso per applicare convenzioni in base al tipo di file, indipendentemente dalla directory, per esempio /*.test.tsx per tutti i file di test;
- preferire le regole specifiche per percorso ai file CLAUDE.md nelle sottodirectory quando le convenzioni

devono valere per file distribuiti nella base di codice.

Obiettivo 3.4: stabilire quando usare la modalità Plan e quando l'esecuzione diretta

Conoscenze:

- la modalità Plan è progettata per attività complesse che comportano modifiche estese, più approcci possibili, decisioni architetturali o modifiche a più file;
- l'esecuzione diretta è adatta a modifiche semplici e ben circoscritte, per esempio aggiungere un controllo di convalida a una funzione;
- la modalità Plan permette di esplorare la base di codice e progettare in sicurezza prima di apportare modifiche, evitando costose rilavorazioni;
- il sottoagente Explore isola l'output esplorativo verboso e restituisce riepiloghi, preservando il contesto della conversazione principale.

Competenze:

- scegliere la modalità Plan per attività con implicazioni architetturali, come riorganizzare microservizi, migrare una libreria che interessa più di 45 file o scegliere tra integrazioni che richiedono infrastrutture diverse;
- scegliere l'esecuzione diretta per modifiche ben comprese e dall'ambito chiaro, come correggere un bug in un singolo file con uno stack trace chiaro o aggiungere una condizione di convalida della data;
- usare il sottoagente Explore nelle fasi esplorative che producono molto output, per evitare di esaurire la finestra di contesto durante attività in più fasi;
- combinare la modalità Plan per l'indagine e l'esecuzione diretta per l'implementazione, per esempio pianificando la migrazione di una libreria prima di realizzare l'approccio scelto.

Obiettivo 3.5: applicare tecniche di perfezionamento iterativo per migliorare progressivamente i risultati

Conoscenze:

- gli esempi concreti di input e output sono il modo più efficace per comunicare le trasformazioni attese quando le descrizioni in prosa vengono interpretate in modo incoerente;
- iterazione guidata dai test: scrivere prima le suite di test e poi perfezionare la soluzione condividendo i test non superati;
- pattern di intervista: chiedere a Claude di porre domande per far emergere aspetti che lo sviluppatore potrebbe non aver considerato, prima dell'implementazione;
- quando presentare tutti i problemi in un unico messaggio, se interagiscono tra loro, e quando risolverli in sequenza, se sono indipendenti.

Competenze:

- fornire 2–3 esempi concreti di input e output per chiarire i requisiti di trasformazione quando le descrizioni in linguaggio naturale producono risultati incoerenti;
- scrivere suite di test che coprano comportamento previsto, casi limite e requisiti di prestazioni prima dell'implementazione, quindi perfezionare la soluzione condividendo i test non superati;
- usare il pattern di intervista per far emergere aspetti di progettazione, come strategie d'invalidazione della cache e modalità di errore, prima di implementare soluzioni in ambiti poco conosciuti;
- fornire casi di test specifici con input di esempio e output atteso per correggere la gestione dei casi limite, come valori nulli negli script di migrazione;
- affrontare in un unico messaggio dettagliato più problemi interdipendenti, ma procedere per iterazioni

successive quando i problemi sono indipendenti.

Obiettivo 3.6: integrare Claude Code nelle pipeline CI/CD

Conoscenze:

- flag -p (o --print) per eseguire Claude Code in modalità non interattiva nelle pipeline automatizzate;
- flag CLI --output-format json e --json-schema per imporre un output strutturato nei flussi CI;
- CLAUDE.md fornisce il contesto del progetto, come standard di test, convenzioni per i fixture e criteri di revisione, quando Claude Code viene invocato in CI;
- isolamento del contesto della sessione: la stessa sessione Claude che ha generato il codice è meno efficace nel rivedere le proprie modifiche rispetto a un'istanza di revisione indipendente.

Competenze:

- eseguire Claude Code in CI con il flag -p per evitare blocchi in attesa di input interattivo;
- usare --output-format json con --json-schema per produrre osservazioni strutturate leggibili dalle macchine, da pubblicare automaticamente come commenti inline sulle pull request;
- includere nel contesto i risultati delle revisioni precedenti quando si ripetono le verifiche dopo nuovi commit, chiedendo a Claude di segnalare solo problemi nuovi o ancora irrisolti per evitare commenti duplicati;
- fornire i file di test esistenti nel contesto, così che la generazione dei test eviti di proporre scenari già coperti dalla suite;
- documentare in CLAUDE.md gli standard di test, i criteri di test più importanti e i fixture disponibili per migliorare la qualità dei test generati e ridurre quelli di scarso valore.

Dominio 4: progettazione dei prompt e output strutturato

Obiettivo 4.1: progettare prompt con criteri espliciti per migliorare la precisione e ridurre i falsi positivi

Conoscenze:

- i criteri espliciti sono più efficaci di indicazioni vaghe, per esempio «segnala un commento solo quando il comportamento dichiarato contraddice quello effettivo del codice» invece di «controlla che i commenti siano corretti»;
- indicazioni generiche come «sii prudente» o «segnala solo osservazioni ad alta affidabilità» non migliorano la precisione quanto criteri categorici specifici;
- un elevato tasso di falsi positivi riduce la fiducia degli sviluppatori: categorie con molti falsi positivi compromettono la fiducia anche nelle categorie accurate.

Competenze:

- scrivere criteri di revisione specifici che stabiliscano quali problemi segnalare (bug, sicurezza) e quali ignorare (stile minore, convenzioni locali), invece di affidarsi a filtri basati sull'affidabilità;
- disattivare temporaneamente le categorie con molti falsi positivi per ripristinare la fiducia degli sviluppatori mentre si migliorano i prompt relativi a tali categorie;
- definire criteri espliciti di gravità con esempi concreti di codice per ogni livello, ottenendo classificazioni coerenti.

Obiettivo 4.2: applicare il prompting few-shot per migliorare coerenza e qualità dell'output

Conoscenze:

- gli esempi few-shot sono la tecnica più efficace per ottenere output coerenti e utilizzabili quando istruzioni dettagliate, da sole, producono risultati variabili;
- gli esempi few-shot mostrano come gestire casi ambigui, per esempio la scelta degli strumenti per richieste ambigue o le lacune nella copertura dei test a livello di branch;
- gli esempi permettono al modello di generalizzare il giudizio a pattern nuovi, invece di limitarsi a riconoscere casi specificati in anticipo;
- gli esempi few-shot possono ridurre le allucinazioni nei compiti di estrazione, per esempio gestendo misure informali e strutture documentali diverse.

Competenze:

- creare 2–4 esempi few-shot mirati per scenari ambigui, mostrando perché è stata scelta una determinata azione invece di alternative plausibili;
- includere esempi che dimostrino il formato di output desiderato (posizione, problema, gravità, correzione suggerita) per ottenere risultati coerenti;
- fornire esempi che distinguano pattern di codice accettabili da problemi reali, riducendo i falsi positivi e favorendo la generalizzazione;
- usare esempi per mostrare come gestire correttamente strutture documentali diverse, come citazioni nel testo rispetto a bibliografie e sezioni metodologiche rispetto a dettagli incorporati nel testo;
- aggiungere esempi di estrazione corretta da documenti con formati diversi per evitare che campi obbligatori risultino vuoti o nulli.

Obiettivo 4.3: imporre output strutturati tramite uso degli strumenti e schemi JSON

Conoscenze:

- l'uso di strumenti (tool_use) con schemi JSON è il metodo più affidabile per ottenere output strutturati conformi allo schema e impedire errori di sintassi JSON;
- differenze tra tool_choice: "auto" (il modello può restituire testo invece di chiamare uno strumento), "any" (deve chiamare uno strumento, ma può sceglierlo) e la selezione forzata (deve chiamare uno specifico strumento nominato);
- gli schemi JSON rigorosi tramite tool_use eliminano gli errori di sintassi, ma non quelli semantici, come righe che non corrispondono al totale o valori nei campi sbagliati;
- aspetti di progettazione degli schemi: campi obbligatori o facoltativi, campi enumerati con pattern "other" più una descrizione dettagliata per categorie estensibili.

Competenze:

- definire strumenti di estrazione con schemi JSON per i parametri di input ed estrarre i dati strutturati dalla risposta tool_use;
- impostare tool_choice: "any" per garantire output strutturato quando sono disponibili più schemi di estrazione e il tipo di documento non è noto;
- forzare uno strumento specifico con tool_choice: {"type":"tool","name":"extract_metadata"} per garantire che una determinata estrazione avvenga prima dei passaggi di arricchimento;
- definire campi facoltativi (nullable) quando le informazioni potrebbero non essere presenti nei documenti di origine, evitando che il modello inventi valori per soddisfare i campi obbligatori;
- aggiungere valori enum come "unclear" per i casi ambigui e campi "other" più dettagli per categorie estensibili;
- affiancare agli schemi di output rigorosi regole di normalizzazione del formato nei prompt, per gestire formattazioni incoerenti nei documenti di origine.

Obiettivo 4.4: implementare cicli di convalida, ripetizione e feedback per la qualità dell'estrazione

Conoscenze:

- ripetere l'operazione fornendo il feedback dell'errore: aggiungere al prompt gli errori specifici di convalida per orientare la correzione;
- i limiti dei tentativi ripetuti: sono inefficaci quando le informazioni richieste non sono presenti nel documento di origine, a differenza degli errori di formato o struttura;
- progettazione dei cicli di feedback: registrare quali costrutti di codice generano osservazioni tramite il campo detected_pattern, per analizzare sistematicamente i pattern delle osservazioni respinte;
- differenza tra errori di convalida semantica (valori che non corrispondono, campo errato) ed errori di sintassi dello schema, eliminati tramite tool_use.

Competenze:

- implementare richieste successive che includano il documento originale, l'estrazione non riuscita e gli errori di convalida specifici, per consentire al modello di correggersi;
- riconoscere quando i tentativi ripetuti sono inutili, per esempio se l'informazione esiste solo in un documento esterno non fornito, e quando invece possono riuscire, come in caso di formati non corrispondenti o errori strutturali nell'output;
- aggiungere il campo detected_pattern alle osservazioni strutturate per analizzare i pattern dei falsi positivi quando gli sviluppatori respingono le segnalazioni;
- progettare flussi di convalida e correzione automatica: estrarre calculated_total accanto a stated_total per segnalare discrepanze e aggiungere booleani conflict_detected per dati incoerenti nella fonte.

Obiettivo 4.5: progettare strategie efficienti di elaborazione in batch

Conoscenze:

- la Message Batches API riduce i costi del 50%, consente un'elaborazione fino a 24 ore e non garantisce un accordo sul livello di servizio (SLA) relativo alla latenza;
- l'elaborazione in batch è adatta a carichi non bloccanti e tolleranti alla latenza, come rapporti notturni, verifiche settimanali e generazione notturna di test, ma non a flussi bloccanti, come controlli prima del merge;
- la Batch API non supporta chiamate agli strumenti multi-turno nella stessa richiesta: non è possibile eseguire strumenti durante la richiesta e restituirne i risultati;
- i campi custom_id associano le richieste e le risposte di un batch.

Competenze:

- scegliere il tipo di API in base alla latenza richiesta: API sincrona per controlli bloccanti prima del merge, Batch API per analisi notturne o settimanali;
- calcolare la frequenza di invio dei batch in base ai vincoli SLA, per esempio finestre di 4 ore per garantire un SLA di 30 ore con elaborazione batch fino a 24 ore;
- gestire i batch non riusciti reinviando solo i documenti falliti, identificati da custom_id, con le opportune modifiche, per esempio suddividendo i documenti che superano i limiti del contesto;
- perfezionare i prompt su un campione prima di elaborare grandi volumi in batch, per aumentare il tasso di successo al primo tentativo e ridurre i costi dei nuovi invii.

Obiettivo 4.6: progettare architetture di revisione multi-istanza e multi-passaggio

Conoscenze:

- limiti dell'autorevisione: nella stessa sessione, il modello conserva il contesto del ragionamento che ha prodotto il risultato ed è meno incline a mettere in discussione le proprie decisioni;
- le istanze di revisione indipendenti, prive del contesto di ragionamento precedente, individuano problemi sottili meglio delle istruzioni di autorevisione o del ragionamento esteso;
- revisione multi-passaggio: suddividere le verifiche ampie in analisi locali per file e passaggi d'integrazione tra file, per evitare dispersione dell'attenzione e osservazioni contraddittorie.

Competenze:

- usare una seconda istanza Claude indipendente per rivedere il codice generato senza il contesto di ragionamento dell'istanza che lo ha prodotto;
- suddividere revisioni ampie su più file in passaggi mirati per individuare problemi locali e passaggi d'integrazione separati per analizzare il flusso dei dati tra file;
- eseguire passaggi di verifica in cui il modello indica il proprio livello di affidabilità accanto a ogni osservazione, così da instradare le revisioni in modo calibrato.

Dominio 5: gestione del contesto e affidabilità

Obiettivo 5.1: gestire il contesto della conversazione per preservare informazioni importanti nelle interazioni lunghe

Conoscenze:

- rischi della sintesi progressiva: valori numerici, percentuali, date e aspettative espresse dai clienti possono essere ridotti a riepiloghi vaghi;
- effetto «lost in the middle»: i modelli elaborano con affidabilità le informazioni all'inizio e alla fine di input lunghi, ma possono omettere risultati dalle sezioni centrali;
- i risultati degli strumenti si accumulano nel contesto e consumano token in misura sproporzionata rispetto alla loro rilevanza, per esempio oltre 40 campi per la ricerca di un ordine quando ne bastano 5;
- per mantenere la coerenza conversazionale, le richieste API successive devono contenere l'intera cronologia della conversazione.

Competenze:

- estrarre i dati transazionali (importi, date, numeri d'ordine, stati) in un blocco persistente di «fatti del caso», incluso in ogni prompt al di fuori della cronologia riassunta;
- estrarre e conservare i dati strutturati sui problemi (ID degli ordini, importi, stati) in un livello di contesto separato per le sessioni con più problemi;
- ridurre l'output verboso degli strumenti ai soli campi pertinenti prima che si accumuli nel contesto, per esempio conservando solo i campi relativi ai resi nei risultati di ricerca degli ordini;
- collocare all'inizio degli input aggregati i riepiloghi dei risultati principali e organizzare i risultati dettagliati con titoli espliciti, per mitigare gli effetti della posizione;
- chiedere ai sottoagenti di includere metadati (date, posizioni delle fonti e contesto metodologico) negli output strutturati per supportare una sintesi successiva accurata;
- modificare gli agenti a monte affinché restituiscano dati strutturati (fatti principali, citazioni, punteggi di pertinenza) invece di contenuti e catene di ragionamento verbosi, quando gli agenti a valle hanno budget di contesto limitati.

Obiettivo 5.2: progettare strategie efficaci per l'escalation e la risoluzione delle ambiguità

Conoscenze:

- condizioni appropriate per l'escalation: richiesta di un operatore da parte del cliente, eccezioni o lacune nelle regole (non solo casi complessi) e impossibilità di compiere progressi significativi;
- differenza tra effettuare subito un'escalation quando il cliente la richiede esplicitamente e offrire una soluzione quando il problema è semplice;
- il sentiment e i punteggi di affidabilità dichiarati dal modello sono indicatori inaffidabili della complessità effettiva del caso;
- più corrispondenze per un cliente richiedono un chiarimento, per esempio chiedendo altri identificatori, anziché una scelta euristica.

Competenze:

- aggiungere al prompt di sistema criteri espliciti per l'escalation ed esempi few-shot che mostrino quando escalare e quando risolvere autonomamente;
- rispettare immediatamente le richieste esplicite di parlare con un operatore, senza tentare prima un'indagine;
- riconoscere la frustrazione e offrire una soluzione quando l'agente può gestire il problema, effettuando l'escalation solo se il cliente ribadisce la propria preferenza;
- effettuare l'escalation quando le regole sono ambigue o non contemplano la richiesta specifica, per esempio l'allineamento al prezzo di un concorrente quando la policy riguarda solo gli adeguamenti dei prezzi sul proprio sito;
- istruire l'agente a chiedere altri identificatori quando gli strumenti restituiscono più corrispondenze, invece di selezionarne una con criteri euristici.

Obiettivo 5.3: implementare strategie di propagazione degli errori nei sistemi multi-agente

Conoscenze:

- un contesto d'errore strutturato (tipo di errore, richiesta tentata, risultati parziali, alternative) permette al coordinatore di decidere come recuperare;
- differenza tra errori di accesso, come i timeout che richiedono una decisione sui nuovi tentativi, e risultati vuoti validi, che indicano una ricerca riuscita senza corrispondenze;
- gli stati d'errore generici, come «ricerca non disponibile», nascondono informazioni utili al coordinatore;
- sopprimere gli errori senza segnalarli, restituendo risultati vuoti come se fossero validi, e interrompere l'intero flusso di lavoro al primo errore sono entrambe pratiche scorrette.

Competenze:

- restituire un contesto d'errore strutturato che includa tipo di errore, operazioni tentate, risultati parziali e possibili alternative, consentendo al coordinatore di decidere come recuperare;
- distinguere gli errori di accesso dai risultati vuoti validi, affinché il coordinatore possa prendere decisioni appropriate;
- fare in modo che i sottoagenti tentino localmente di recuperare dagli errori temporanei e trasmettano solo quelli non risolvibili, includendo operazioni tentate e risultati parziali;
- strutturare la sintesi con annotazioni sulla copertura che indichino quali risultati sono ben supportati e quali aree presentano lacune dovute a fonti non disponibili.

Obiettivo 5.4: gestire il contesto durante l'esplorazione di basi di codice ampie

Conoscenze:

- nelle sessioni prolungate il contesto può degradarsi: i modelli iniziano a fornire risposte incoerenti e a fare riferimento a pattern «tipici» anziché alle classi specifiche individuate in precedenza;
- i file scratchpad permettono di conservare i risultati principali oltre i confini del contesto;
- la delega ai sottoagenti isola gli output esplorativi verbosi, mentre l'agente principale coordina la comprensione generale;
- la persistenza strutturata dello stato consente il ripristino dopo un arresto anomalo: ogni agente esporta lo stato in un percorso noto e il coordinatore carica un manifest alla ripresa.

Competenze:

- creare sottoagenti per indagare quesiti specifici, per esempio «trova tutti i file di test» o «traccia le dipendenze del flusso dei rimborsi», mentre l'agente principale mantiene il coordinamento generale;
- chiedere agli agenti di mantenere file scratchpad con i risultati principali e consultarli per le domande successive, contrastando il degrado del contesto;
- riassumere i risultati principali di una fase di esplorazione prima di avviare i sottoagenti per la fase seguente, inserendo i riepiloghi nel contesto iniziale;
- progettare il ripristino dopo un arresto anomalo tramite esportazioni strutturate dello stato degli agenti (manifest), che il coordinatore carica alla ripresa e inserisce nei prompt degli agenti;
- usare /compact per ridurre l'uso del contesto durante sessioni di esplorazione prolungate, quando l'output esplorativo verboso occupa troppo spazio.

Obiettivo 5.5: progettare flussi di revisione umana e calibrare i livelli di affidabilità

Conoscenze:

- metriche aggregate di accuratezza, per esempio il 97% complessivo, possono nascondere prestazioni scarse per specifici tipi di documenti o campi;
- il campionamento casuale stratificato misura i tassi di errore nelle estrazioni ad alta affidabilità e individua nuovi pattern di errore;
- i punteggi di affidabilità a livello di campo, calibrati con insiemi di convalida etichettati, consentono di indirizzare l'attenzione dei revisori;
- prima di automatizzare le estrazioni ad alta affidabilità è importante convalidare l'accuratezza per tipo di documento e per segmento di campo.

Competenze:

- implementare un campionamento casuale stratificato delle estrazioni ad alta affidabilità per misurare continuamente gli errori e individuare nuovi pattern;
- analizzare l'accuratezza per tipo di documento e campo per verificare prestazioni coerenti in tutti i segmenti prima di ridurre la revisione umana;
- chiedere ai modelli punteggi di affidabilità a livello di campo e poi calibrare le soglie di revisione usando insiemi di convalida etichettati;
- indirizzare alla revisione umana le estrazioni con bassa affidabilità del modello o con fonti ambigue o contraddittorie, assegnando priorità alla capacità limitata dei revisori.

Obiettivo 5.6: preservare la provenienza delle informazioni e gestire l'incertezza nelle sintesi basate su più fonti

Conoscenze:

- durante la sintesi si può perdere l'attribuzione delle fonti se i risultati vengono compressi senza conservare la corrispondenza tra affermazioni e fonti;
- è importante usare corrispondenze strutturate tra affermazioni e fonti, che l'agente di sintesi deve conservare e unire quando combina i risultati;
- in presenza di statistiche discordanti provenienti da fonti affidabili, occorre annotare i conflitti e attribuire le fonti, anziché scegliere arbitrariamente un valore;
- per interpretare correttamente i dati temporali, gli output strutturati devono riportare le date di pubblicazione o raccolta, evitando che differenze temporali vengano interpretate come contraddizioni.

Competenze:

- chiedere ai sottoagenti di restituire corrispondenze strutturate tra affermazioni e fonti (URL, nomi dei documenti, estratti pertinenti), da conservare nei passaggi successivi di sintesi;
- strutturare i rapporti con sezioni distinte per risultati consolidati e controversi, conservando le descrizioni originali delle fonti e il contesto metodologico;
- completare l'analisi documentale includendo e annotando esplicitamente i valori discordanti, lasciando al coordinatore la decisione su come conciliarli prima della sintesi;
- chiedere ai sottoagenti di includere negli output strutturati le date di pubblicazione o di raccolta dei dati per consentire una corretta interpretazione temporale;
- rappresentare i diversi tipi di contenuti nel formato più adatto durante la sintesi: dati finanziari in tabelle, notizie in prosa e risultati tecnici in elenchi strutturati, invece di uniformare tutto.

7. Come prepararsi

Per prepararti all'esame di certificazione:

- Crea un agente con Claude Agent SDK: implementa un loop agentico completo con chiamate agli strumenti, gestione degli errori e delle sessioni. Esercitati a creare sottoagenti e a passare loro il contesto.
- Configura Claude Code per un progetto reale: imposta una gerarchia di configurazione in CLAUDE.md, crea regole specifiche per percorso in .claude/rules/, definisci skill personalizzate con opzioni frontmatter (context: fork, allowed-tools) e integra almeno un server MCP.
- Progetta e verifica strumenti MCP: scrivi descrizioni che distinguano chiaramente strumenti simili. Implementa risposte d'errore strutturate con categorie e flag di ripetibilità. Verifica l'affidabilità della scelta degli strumenti con richieste ambigue.
- Crea una pipeline per l'estrazione di dati strutturati: usa tool_use con schemi JSON, implementa cicli di convalida e ripetizione, progetta schemi con campi facoltativi o nullable ed esercitati con l'elaborazione batch tramite Message Batches API.
- Esercitati nelle tecniche di progettazione dei prompt: scrivi esempi few-shot per scenari ambigui. Definisci criteri di revisione espliciti per ridurre i falsi positivi. Progetta architetture di revisione multi-passaggio per revisioni ampie del codice.
- Studia la gestione del contesto: esercitati a estrarre fatti strutturati da output verbosi degli strumenti, a usare file scratchpad nelle sessioni lunghe e a delegare ai sottoagenti per gestire i limiti del contesto.
- Ripassa le strategie di escalation e con intervento umano: comprendi quando effettuare l'escalation

(lacune nelle regole, richieste dei clienti, impossibilità di procedere) e quando risolvere autonomamente. Esercitati a progettare flussi di revisione umana con instradamento basato sull'affidabilità.

8. Esercizi di preparazione

Completa questi esercizi pratici per acquisire familiarità con gli argomenti dell'esame. Ciascun esercizio consolida le conoscenze di uno o più domini.

Esercizio 1: crea un agente con più strumenti e logica di escalation

Obiettivo: esercitarsi a progettare un loop agentico con integrazione degli strumenti, gestione strutturata degli errori e strategie di escalation.

Passaggi:

- definisci 3–4 strumenti MCP con descrizioni dettagliate che distinguano chiaramente lo scopo, gli input previsti e i limiti di ciascuno. Includi almeno due strumenti con funzioni simili, che richiedano descrizioni precise per evitare confusioni nella selezione;
- implementa un loop agentico che controlli stop_reason per decidere se continuare a eseguire strumenti o presentare la risposta finale. Gestisci correttamente sia tool_use sia end_turn;
- aggiungi agli strumenti risposte d'errore strutturate con errorCategory (transient, validation, permission), un booleano isRetryable e descrizioni comprensibili. Verifica che l'agente gestisca correttamente ogni tipo di errore, ripetendo le operazioni dopo errori temporanei e spiegando all'utente gli errori aziendali;
- implementa un hook programmatico che intercetti le chiamate agli strumenti per applicare una regola aziendale, per esempio bloccare operazioni oltre una determinata soglia e indirizzarle verso un flusso di escalation;
- esegui test con messaggi che includono più problemi e verifica che l'agente li suddivida, gestisca ogni aspetto e produca una risposta unificata.

Domini consolidati: Dominio 1 (architettura agentica e orchestrazione), Dominio 2 (progettazione degli strumenti e integrazione MCP), Dominio 5 (gestione del contesto e affidabilità).

Esercizio 2: configura Claude Code per il flusso di sviluppo di un team

Obiettivo: esercitarsi a configurare gerarchie CLAUDE.md, comandi slash personalizzati, regole specifiche per percorso e integrazione MCP in un progetto con più sviluppatori.

Passaggi:

- crea un CLAUDE.md a livello di progetto con standard di programmazione universali e convenzioni di test. Verifica che le istruzioni a livello di progetto vengano applicate in modo coerente a tutti i membri del team;
- crea file .claude/rules/ con pattern glob nel frontmatter YAML per diverse aree del codice, per esempio paths: ["src/api//"] per le convenzioni API e paths: ["/.test.*"] per quelle dei test. Verifica che le regole vengano caricate solo quando si modificano file corrispondenti;
- crea una skill a livello di progetto in .claude/skills/ con context: fork e restrizioni allowed-tools. Verifica che venga eseguita in isolamento, senza contaminare il contesto della conversazione principale;
- configura un server MCP in .mcp.json con espansione delle variabili d'ambiente per le credenziali. Aggiungi un server MCP sperimentale personale in ~/.claude.json e verifica che entrambi siano disponibili contemporaneamente;
- confronta la modalità Plan e l'esecuzione diretta su attività di complessità diversa: correzione di un bug

in un singolo file, migrazione di una libreria con più file e nuova funzionalità con più approcci implementativi validi. Valuta quando la modalità Plan è utile.

Domini consolidati: Dominio 3 (configurazione e flussi di lavoro di Claude Code), Dominio 2 (progettazione degli strumenti e integrazione MCP).

Esercizio 3: crea una pipeline per l'estrazione di dati strutturati

Obiettivo: esercitarsi a progettare schemi JSON, usare tool_use per ottenere output strutturati, implementare cicli di convalida e ripetizione e pianificare l'elaborazione batch.

Passaggi:

- definisci uno strumento di estrazione con uno schema JSON che contenga campi obbligatori e facoltativi, un enum con il pattern "other" più un campo descrittivo e campi nullable per informazioni che potrebbero non esistere nei documenti di origine. Elabora documenti in cui mancano alcuni campi e verifica che il modello restituisca null invece di inventare valori;
- implementa un ciclo di convalida e ripetizione: se la convalida Pydantic o dello schema JSON non riesce, invia una richiesta successiva che includa documento, estrazione non riuscita ed errore di convalida specifico. Tieni traccia degli errori risolvibili con un nuovo tentativo, come i problemi di formato, e di quelli non risolvibili, come le informazioni assenti dalla fonte;
- aggiungi esempi few-shot che mostrino l'estrazione da documenti in formati diversi, per esempio citazioni nel testo e bibliografie, descrizioni narrative e tabelle strutturate, e verifica che migliorino la gestione delle diverse strutture;
- progetta una strategia batch: invia 100 documenti tramite Message Batches API, gestisci gli errori usando custom_id, invia nuovamente i documenti non riusciti con le modifiche necessarie, come la suddivisione dei documenti troppo grandi, e calcola il tempo totale rispetto ai vincoli SLA;
- implementa una strategia di instradamento alla revisione umana: chiedi al modello punteggi di affidabilità per ogni campo, invia alla revisione umana le estrazioni poco affidabili e analizza l'accuratezza per tipo di documento e campo per verificarne la coerenza.

Domini consolidati: Dominio 4 (progettazione dei prompt e output strutturato), Dominio 5 (gestione del contesto e affidabilità).

Esercizio 4: progetta ed esegui il debug di una pipeline di ricerca multi-agente

Obiettivo: esercitarsi a orchestrare sottoagenti, passare loro il contesto, propagare gli errori e sintetizzare informazioni preservandone la provenienza.

Passaggi:

- crea un agente coordinatore che deleghi il lavoro ad almeno due sottoagenti, per esempio uno per la ricerca web e uno per l'analisi documentale. Verifica che allowedTools del coordinatore includa Task e che ogni sottoagente riceva direttamente nel proprio prompt i risultati della ricerca, senza fare affidamento sull'ereditarietà automatica del contesto;
- implementa l'esecuzione parallela dei sottoagenti facendo emettere al coordinatore più chiamate allo strumento Task in un'unica risposta. Misura il miglioramento della latenza rispetto all'esecuzione sequenziale;
- definisci output strutturati per i sottoagenti che separino contenuto e metadati: ogni risultato deve includere un'affermazione, un estratto probatorio, l'URL o il nome del documento di origine e la data di pubblicazione. Verifica che il sottoagente di sintesi conservi l'attribuzione delle fonti;
-

implementa la propagazione degli errori: simula un timeout di un sottoagente e verifica che il coordinatore riceva un contesto d'errore strutturato (tipo di errore, richiesta tentata, risultati parziali). Verifica che possa procedere con i risultati parziali e annotare le lacune di copertura nell'output finale;
- esegui test con dati discordanti tra le fonti, per esempio statistiche diverse in due fonti affidabili, e verifica che la sintesi conservi entrambi i valori con l'attribuzione delle fonti, senza sceglierne uno arbitrariamente, distinguendo i risultati consolidati da quelli controversi.

Domini consolidati: Dominio 1 (architettura agentica e orchestrazione), Dominio 2 (progettazione degli strumenti e integrazione MCP), Dominio 5 (gestione del contesto e affidabilità).

9. Domande di esempio

Le domande seguenti illustrano formato e difficoltà dell'esame. Sono tratte dal test di esercitazione e includono spiegazioni a scopo didattico.

Scenario: agente per la risoluzione delle richieste di assistenza clienti

Domanda 1

I dati di produzione mostrano che nel 12% dei casi l'agente salta completamente get_customer e chiama lookup_order usando solo il nome dichiarato dal cliente, causando talvolta l'identificazione errata dell'account e rimborsi non corretti. Quale modifica risolverebbe il problema di affidabilità in modo più efficace?

- A. Aggiungere un prerequisito programmatico che blocchi le chiamate a lookup_order e process_refund finché get_customer non restituisce un ID cliente verificato.
- B. Rafforzare il prompt di sistema specificando che la verifica del cliente tramite get_customer è obbligatoria prima di qualsiasi operazione sugli ordini.
- C. Aggiungere esempi few-shot che mostrino l'agente mentre chiama sempre prima get_customer, anche quando i clienti forniscono spontaneamente i dettagli dell'ordine.
- D. Implementare un classificatore di instradamento che analizzi ogni richiesta e abiliti solo il sottoinsieme di strumenti appropriato per quel tipo di richiesta.

Risposta corretta: A. Quando una sequenza specifica di strumenti è necessaria per una logica aziendale critica, come verificare l'identità del cliente prima di elaborare un rimborso, l'applicazione programmatica dei vincoli garantisce un comportamento deterministico, cosa che gli approcci basati sui prompt non possono garantire. Le opzioni B e C si affidano alla conformità probabilistica del modello, insufficiente quando gli errori hanno conseguenze finanziarie. L'opzione D riguarda la disponibilità degli strumenti, non il loro ordine d'uso, che è il vero problema.

Domanda 2

I log di produzione mostrano che l'agente chiama spesso get_customer quando gli utenti chiedono informazioni sugli ordini, per esempio «controlla il mio ordine n. 12345», invece di chiamare lookup_order. Entrambi gli strumenti hanno descrizioni minime («Recupera informazioni sul cliente» e «Recupera dettagli dell'ordine») e accettano identificatori simili. Qual è il primo intervento più efficace per migliorare l'affidabilità della selezione?

- A. Aggiungere al prompt di sistema esempi few-shot che mostrino i pattern corretti di selezione degli strumenti, includendo 5–8 esempi di richieste relative agli ordini instradate a lookup_order.

- B. Ampliare la descrizione di ciascuno strumento includendo i formati di input gestiti, richieste di esempio, casi limite e indicazioni su quando usarlo rispetto a strumenti simili.
- C. Implementare un livello di instradamento che analizzi l'input dell'utente prima di ogni turno e preselezioni lo strumento appropriato in base a parole chiave e pattern degli identificatori.
- D. Consolidare i due strumenti in un unico lookup_entity, che accetti qualsiasi identificatore e determini internamente quale backend interrogare.

Risposta corretta: B. Le descrizioni degli strumenti sono il meccanismo principale con cui i modelli linguistici scelgono gli strumenti. Quando sono troppo brevi, il modello non dispone del contesto necessario per distinguere strumenti simili. L'opzione B affronta direttamente la causa alla radice con poco sforzo e un impatto elevato. Gli esempi few-shot (A) aumentano l'uso di token senza correggere il problema di fondo. Un livello di instradamento (C) è eccessivamente complesso e aggira la comprensione del linguaggio naturale del modello. Consolidare gli strumenti (D) è una scelta architetturale valida, ma richiede più lavoro di quanto sia opportuno per un primo intervento quando il problema immediato è la descrizione inadeguata.

Domanda 3

L'agente risolve al primo contatto il 55% delle richieste, molto meno dell'obiettivo dell'80%. I log mostrano che effettua l'escalation di casi semplici, come sostituzioni standard per prodotti danneggiati corredate da foto, ma tenta di gestire autonomamente situazioni complesse che richiedono eccezioni alle regole. Qual è il modo più efficace per migliorare la calibrazione delle escalation?

- A. Aggiungere al prompt di sistema criteri espliciti per l'escalation ed esempi few-shot che mostrino quando effettuarla e quando risolvere autonomamente.
- B. Chiedere all'agente di dichiarare un punteggio di affidabilità da 1 a 10 prima di ogni risposta e instradare automaticamente le richieste a operatori umani quando il punteggio è inferiore a una soglia.
- C. Distribuire un modello classificatore separato, addestrato sui ticket storici, per prevedere quali richieste richiedano un'escalation prima che l'agente principale inizi l'elaborazione.
- D. Implementare l'analisi del sentiment per rilevare la frustrazione del cliente ed effettuare automaticamente l'escalation quando il sentiment negativo supera una soglia.

Risposta corretta: A. Criteri espliciti per l'escalation, accompagnati da esempi few-shot, affrontano direttamente la causa principale: confini decisionali poco chiari. È l'intervento iniziale proporzionato. L'opzione B non funziona perché i punteggi di affidabilità dichiarati dal modello sono scarsamente calibrati: l'agente è già eccessivamente sicuro nei casi difficili. L'opzione C è eccessivamente complessa: richiede dati etichettati e infrastruttura di machine learning, senza aver prima provato a migliorare i prompt. L'opzione D affronta un problema diverso: il sentiment non è correlato alla complessità del caso, che è il vero problema.

Scenario: generazione di codice con Claude Code

Domanda 4

Vuoi creare un comando slash personalizzato /review che esegua la checklist standard di revisione del codice del team. Il comando deve essere disponibile a ogni sviluppatore che clona o aggiorna il repository. In quale percorso devi creare il file del comando?

- A. Nella directory .claude/commands/ del repository di progetto.
- B. In ~/.claude/commands/ nella directory home di ogni sviluppatore.

- C. Nel file CLAUDE.md alla radice del progetto.
- D. In un file .claude/config.json con un array commands.

Risposta corretta: A. I comandi slash personalizzati a livello di progetto devono essere salvati nella directory .claude/commands/ del repository. Sono gestiti dal controllo versione e diventano automaticamente disponibili a tutti gli sviluppatori quando clonano o aggiornano il repository. L'opzione B (~/.claude/commands/) è riservata ai comandi personali, non condivisi tramite controllo versione. L'opzione C (CLAUDE.md) serve per istruzioni e contesto del progetto, non per definire comandi. L'opzione D descrive un meccanismo di configurazione inesistente in Claude Code.

Domanda 5

Ti è stato assegnato il compito di ristrutturare l'applicazione monolitica del team trasformandola in microservizi. Sarà necessario modificare decine di file e prendere decisioni sui confini dei servizi e sulle dipendenze tra moduli. Quale approccio dovresti adottare?

- A. Attivare la modalità Plan per esplorare la base di codice, comprendere le dipendenze e progettare un approccio implementativo prima di apportare modifiche.
- B. Iniziare con l'esecuzione diretta e procedere per modifiche incrementali, lasciando che sia l'implementazione a rivelare i confini naturali dei servizi.
- C. Usare l'esecuzione diretta con istruzioni iniziali dettagliate che specifichino esattamente come strutturare ogni servizio.
- D. Iniziare in modalità di esecuzione diretta e passare alla modalità Plan solo se durante l'implementazione emergono complessità impreviste.

Risposta corretta: A. La modalità Plan è progettata per attività complesse con modifiche estese, più approcci validi e decisioni architetturali: è esattamente il caso della ristrutturazione da monolite a microservizi. Consente di esplorare e progettare in sicurezza prima di modificare il codice. L'opzione B rischia di causare costose rilavorazioni quando le dipendenze emergono in ritardo. L'opzione C presume che la struttura corretta sia già nota, senza aver esplorato il codice. L'opzione D ignora che la complessità è già esplicita nei requisiti e non è un'eventualità futura.

Domanda 6

La tua base di codice ha aree distinte con convenzioni di programmazione diverse: i componenti React usano uno stile funzionale con hook, i gestori API usano async/await e specifiche modalità di gestione degli errori, mentre i modelli di database seguono il pattern repository. I file di test sono distribuiti nella base di codice accanto al codice verificato, per esempio Button.test.tsx accanto a Button.tsx, e vuoi applicare le stesse convenzioni di test ovunque si trovino. Qual è il metodo più manutenibile per fare in modo che Claude applichi automaticamente le convenzioni corrette durante la generazione del codice?

- A. Creare file di regole in .claude/rules/ con pattern glob nel frontmatter YAML, per applicare condizionalmente le convenzioni in base ai percorsi dei file.
- B. Consolidare tutte le convenzioni nel file CLAUDE.md alla radice, organizzandole per area e affidandosi a Claude per dedurre quale sezione applicare.
- C. Creare skill in .claude/skills/ per ciascun tipo di codice, inserendo le convenzioni pertinenti nei rispettivi file SKILL.md.
- D. Inserire un file CLAUDE.md separato in ogni sottodirectory con le convenzioni specifiche dell'area.

Risposta corretta: A. L'opzione A è corretta perché .claude/rules/ con pattern glob, come /*.test.tsx, applica automaticamente le convenzioni in base ai percorsi dei file, indipendentemente dalla directory,

caratteristica essenziale per i file di test distribuiti nella base di codice. L'opzione B si affida a un'inferenza anziché a una corrispondenza esplicita, risultando inaffidabile. L'opzione C richiede l'invocazione manuale della skill o si affida alla scelta di Claude di caricarla, contraddicendo la necessità di un'applicazione automatica e deterministica in base ai percorsi. L'opzione D non gestisce facilmente i file distribuiti in molte directory, poiché i file CLAUDE.md sono legati alla directory che li contiene.

Scenario: sistema di ricerca multi-agente

Domanda 7

Dopo aver eseguito il sistema sull'argomento «impatto dell'IA sulle industrie creative», osservi che ogni sottoagente completa correttamente il proprio compito: l'agente di ricerca web trova articoli pertinenti, quello di analisi documentale riassume correttamente gli studi e quello di sintesi produce un risultato coerente. Tuttavia, i rapporti finali trattano solo le arti visive, ignorando completamente musica, scrittura e produzione cinematografica. Esaminando i log del coordinatore, noti che ha suddiviso l'argomento in tre attività: «IA nella creazione di arte digitale», «IA nella grafica» e «IA nella fotografia». Qual è la causa principale più probabile?

- A. L'agente di sintesi non ha istruzioni per individuare lacune di copertura nei risultati ricevuti dagli altri agenti.
- B. La scomposizione delle attività del coordinatore è troppo circoscritta e assegna ai sottoagenti compiti che non coprono tutti gli ambiti pertinenti dell'argomento.
- C. Le richieste dell'agente di ricerca web non sono abbastanza complete e devono essere ampliate per includere altri settori creativi.
- D. L'agente di analisi documentale esclude fonti relative ai settori creativi non visivi a causa di criteri di pertinenza troppo restrittivi.

Risposta corretta: B. I log del coordinatore rivelano direttamente la causa principale: ha suddiviso il tema delle «industrie creative» in attività che riguardano solo le arti visive (arte digitale, grafica e fotografia), omettendo del tutto musica, scrittura e cinema. I sottoagenti hanno svolto correttamente le attività assegnate: il problema è l'assegnazione stessa. Le opzioni A, C e D attribuiscono erroneamente la colpa ad agenti a valle che operano correttamente entro l'ambito ricevuto.

Domanda 8

Il sottoagente di ricerca web va in timeout mentre indaga un argomento complesso. Devi progettare come comunicare questo errore all'agente coordinatore. Quale strategia di propagazione degli errori permette il recupero più efficace?

- A. Restituire al coordinatore un contesto d'errore strutturato che includa tipo di errore, richiesta tentata, eventuali risultati parziali e possibili approcci alternativi.
- B. Implementare nel sottoagente tentativi automatici con backoff esponenziale e restituire uno stato generico «ricerca non disponibile» solo dopo aver esaurito i tentativi.
- C. Intercettare il timeout nel sottoagente e restituire un insieme di risultati vuoto, indicandolo come riuscito.
- D. Propagare direttamente l'eccezione di timeout a un gestore di livello superiore che interrompa l'intero flusso di ricerca.

Risposta corretta: A. Un contesto d'errore strutturato fornisce al coordinatore le informazioni necessarie

per scegliere come recuperare: riprovare con una richiesta modificata, adottare un approccio alternativo o procedere con i risultati parziali. Lo stato generico dell'opzione B nasconde informazioni utili e impedisce decisioni consapevoli. L'opzione C sopprime l'errore presentando il fallimento come un successo, impedendo il recupero e rischiando di produrre risultati di ricerca incompleti. L'opzione D interrompe inutilmente l'intero flusso di lavoro, anche quando sarebbero possibili strategie di recupero.

Domanda 9

Durante i test osservi che l'agente di sintesi deve spesso verificare affermazioni specifiche mentre combina i risultati. Attualmente, quando serve una verifica, l'agente di sintesi restituisce il controllo al coordinatore, che invoca l'agente di ricerca web e poi richiama la sintesi con i risultati. Questo aggiunge 2–3 passaggi per attività e aumenta la latenza del 40%. La valutazione mostra che l'85% delle verifiche riguarda semplici controlli dei fatti (date, nomi, statistiche), mentre il 15% richiede indagini più approfondite. Qual è il metodo più efficace per ridurre l'overhead mantenendo l'affidabilità del sistema?

- A. Fornire all'agente di sintesi uno strumento circoscritto verify_fact per le ricerche semplici, continuando a delegare tramite il coordinatore le verifiche complesse all'agente di ricerca web.
- B. Far raccogliere all'agente di sintesi tutte le verifiche necessarie e restituirle in blocco al coordinatore al termine del passaggio, affinché le invii insieme all'agente di ricerca web.
- C. Fornire all'agente di sintesi tutti gli strumenti di ricerca web, affinché possa gestire direttamente qualsiasi verifica senza passare dal coordinatore.
- D. Chiedere all'agente di ricerca web di memorizzare in anticipo informazioni supplementari su ogni fonte iniziale, anticipando ciò che l'agente di sintesi potrebbe dover verificare.

Risposta corretta: A. L'opzione A applica il principio del privilegio minimo: assegna all'agente di sintesi solo ciò che serve per il caso frequente, l'85% delle verifiche semplici, e conserva il modello di coordinamento attuale per i casi complessi. L'opzione B crea dipendenze bloccanti, poiché i passaggi della sintesi potrebbero dipendere da fatti verificati in precedenza. L'opzione C fornisce all'agente di sintesi privilegi eccessivi, violando la separazione delle responsabilità. L'opzione D si affida a una memorizzazione speculativa che non può prevedere in modo affidabile quali verifiche serviranno.

Scenario: Claude Code per l'integrazione continua

Domanda 10

Lo script della pipeline esegue claude "Analyze this pull request for security issues", ma il job si blocca indefinitamente. I log indicano che Claude Code è in attesa di input interattivo. Qual è il modo corretto per eseguire Claude Code in una pipeline automatizzata?

- A. Aggiungere il flag -p: claude -p "Analyze this pull request for security issues".
- B. Impostare la variabile d'ambiente CLAUDE_HEADLESS=true prima di eseguire il comando.
- C. Reindirizzare lo standard input da /dev/null: claude "Analyze this pull request for security issues" < /dev/null.
- D. Aggiungere il flag --batch: claude --batch "Analyze this pull request for security issues".

Risposta corretta: A. Il flag -p (o --print) è il metodo documentato per eseguire Claude Code in modalità non interattiva. Elabora il prompt, scrive il risultato sullo standard output e termina senza attendere input: esattamente ciò che serve nelle pipeline CI/CD. Le altre opzioni fanno riferimento a funzionalità inesistenti (CLAUDE_HEADLESS e --batch) oppure usano soluzioni alternative Unix che non risolvono correttamente la sintassi del comando Claude Code.

Domanda 11

Il tuo team vuole ridurre i costi API per l'analisi automatizzata. Attualmente, chiamate Claude in tempo reale alimentano due flussi di lavoro: (1) un controllo bloccante prima del merge, che deve terminare prima che gli sviluppatori possano effettuare il merge, e (2) un rapporto sul debito tecnico generato durante la notte e consultato la mattina successiva. Il responsabile propone di passare per entrambi alla Message Batches API per risparmiare il 50% sui costi. Come valuteresti la proposta?

- A. Usare l'elaborazione batch solo per i rapporti sul debito tecnico e mantenere le chiamate in tempo reale per i controlli prima del merge.
- B. Passare entrambi i flussi all'elaborazione batch e controllare lo stato per verificarne il completamento.
- C. Mantenere le chiamate in tempo reale per entrambi i flussi, per evitare problemi nell'ordinamento dei risultati batch.
- D. Passare entrambi i flussi all'elaborazione batch con un fallback a tempo scaduto che avvii una chiamata in tempo reale se i batch richiedono troppo tempo.

Risposta corretta: A. La Message Batches API consente di risparmiare il 50%, ma può impiegare fino a 24 ore e non garantisce un SLA sulla latenza. Non è quindi adatta ai controlli bloccanti prima del merge, che costringono gli sviluppatori ad attendere il risultato, ma è ideale per lavori notturni come i rapporti sul debito tecnico. L'opzione B è errata perché non è accettabile fare affidamento su un completamento «spesso più rapido» per flussi bloccanti. L'opzione C si basa su un equivoco: i risultati batch possono essere associati tramite i campi custom_id. L'opzione D aggiunge complessità non necessaria, quando la soluzione più semplice è scegliere l'API adatta a ciascun caso.

Domanda 12

Una pull request modifica 14 file del modulo di tracciamento delle scorte. La revisione in un unico passaggio di tutti i file produce risultati incoerenti: feedback dettagliato su alcuni file e superficiale su altri, bug evidenti non individuati e osservazioni contraddittorie, con lo stesso pattern segnalato come problematico in un file e approvato in un altro. Come dovresti ristrutturare la revisione?

- A. Suddividerla in passaggi mirati: analizzare ogni file singolarmente per i problemi locali, poi eseguire un passaggio d'integrazione separato per esaminare il flusso dei dati tra file.
- B. Chiedere agli sviluppatori di suddividere le pull request ampie in invii più piccoli, di 3–4 file, prima della revisione automatizzata.
- C. Passare a un modello di livello superiore con una finestra di contesto più ampia, per esaminare adeguatamente tutti i 14 file in un unico passaggio.
- D. Eseguire tre revisioni indipendenti dell'intera pull request e segnalare solo i problemi rilevati in almeno due delle tre.

Risposta corretta: A. Suddividere la revisione in passaggi mirati affronta direttamente la causa principale: la dispersione dell'attenzione quando si elaborano molti file insieme. L'analisi file per file garantisce una profondità coerente, mentre il passaggio d'integrazione separato rileva i problemi tra file. L'opzione B trasferisce il carico agli sviluppatori senza migliorare il sistema. L'opzione C presume erroneamente che una finestra di contesto più ampia risolva i problemi di qualità dell'attenzione. L'opzione D rischia di non rilevare bug reali, richiedendo un consenso su problemi che potrebbero essere individuati solo in alcune esecuzioni.

10. Valutazione dell'esame

L'esame Claude Certified Architect – Foundations è una valutazione basata su criteri: ogni candidato viene confrontato con uno standard di prestazione fisso, non con gli altri candidati. Per superarlo occorre dimostrare le conoscenze e le competenze definite nel blueprint, non ottenere un risultato superiore a una determinata percentuale degli altri partecipanti.

Soglia di superamento. Il punteggio minimo è stato stabilito tramite uno studio formale, nel quale esperti qualificati hanno definito il livello di prestazione atteso da un candidato minimamente qualificato. Il punteggio viene riportato su una scala ponderata da 100 a 1.000; la soglia di superamento è 720. I modelli di punteggio ponderato consentono di confrontare i risultati di versioni diverse dell'esame, che potrebbero avere livelli di difficoltà leggermente differenti.

Comunicazione dei risultati. Il risultato indica se l'esame è stato superato, insieme a un punteggio ponderato da 100 a 1.000. Il rapporto mostra inoltre la percentuale di domande corrette per ogni dominio. Le percentuali per sezione aiutano a comprendere la propria prestazione, ma non determinano l'esito: il superamento dipende dal punteggio ponderato totale.

11. Registrazione e prenotazione

La registrazione e la prenotazione avvengono tramite Anthropic Partner Academy e Pearson VUE:

1. Vai alla pagina di certificazione dell'esame su Anthropic Partner Academy e consulta i dettagli.
2. Scarica la guida all'esame e leggi i Termini e condizioni della certificazione e le Regole per gli esami di certificazione prima di registrarti.
3. Registrati all'esame e completa il pagamento. L'importo mostrato al checkout tiene conto degli eventuali sconti previsti dal tuo livello di partner.
4. Segui le istruzioni di conferma per creare un account Pearson VUE, quindi accedi per prenotare la sessione d'esame.
5. Scegli una data disponibile e seleziona la sorveglianza online oppure un centro d'esame Pearson. L'esame è erogato da Pearson VUE.
6. Puoi annullare o riprogrammare l'esame fino a 24 ore prima dell'appuntamento. Le modifiche effettuate nelle 24 ore precedenti comportano la perdita della quota d'esame.

12. Regole dell'esame

Documento d'identità

Il giorno dell'esame devi presentare un documento d'identità valido, non scaduto, rilasciato da un ente governativo e munito di fotografia. Il nome sul documento deve corrispondere esattamente a quello della registrazione. Per correggere il nome registrato, contatta certifications-support@anthropic.com prima di prenotare l'esame.

Misure di accomodamento

Sono disponibili misure di accomodamento ragionevoli per i candidati con disabilità o esigenze documentate, in conformità alla legge applicabile. Pearson VUE deve approvare le richieste prima della

prenotazione. Non prenotare l'appuntamento finché la richiesta non è stata approvata. Le richieste si presentano su pearsonvue.com/us/en/test-takers/accommodations.

Politica sui tentativi successivi

I candidati che non superano l'esame possono ripeterlo dopo un periodo di attesa obbligatorio, che aumenta a ogni tentativo non superato: 14 giorni dopo il primo, 30 giorni dopo il secondo e 90 giorni dopo il terzo. È possibile sostenere lo stesso esame fino a quattro volte nell'arco di dodici mesi. I limiti si applicano a ciascun esame: non superare un esame non impedisce di iscriversi a un esame diverso. La quota è dovuta per ogni tentativo.

Assenza e ritardo

Chi non si presenta all'esame prenotato o arriva oltre il periodo di tolleranza consentito perde la quota d'esame e deve registrarsi nuovamente. Le scadenze per annullare o riprogrammare sono descritte nella sezione 11.

13. Svolgimento dell'esame e regole di condotta

L'esame si svolge in un ambiente standardizzato, sicuro e sorvegliato. Le regole seguenti valgono sia per gli esami online sia per quelli presso un centro Pearson VUE e tutelano l'integrità della certificazione per tutti i titolari.

Durante l'esame è necessario:

- rimanere sempre nel campo visivo del supervisore e della webcam, se l'esame è online;
- tenere l'area di lavoro libera da appunti, libri, telefoni, monitor secondari e altri materiali;
- non comunicare con altre persone;
- non acquisire, copiare, fotografare o riprodurre in alcun modo i contenuti dell'esame.

Tra gli oggetti vietati rientrano telefoni cellulari, smartwatch, cuffie, materiali di studio e dispositivi di registrazione. Gli eventuali oggetti consentiti, come la carta per appunti fornita dal supervisore, sono specificati da Pearson VUE.

Conseguenze della condotta scorretta. Barare, tentare di accedere a risorse vietate o divulgare contenuti dell'esame può comportare l'annullamento del risultato, la revoca della credenziale e l'esclusione da esami futuri.

14. Riservatezza e accordo di non divulgazione

Prima dell'inizio dell'esame devi accettare un accordo di riservatezza e non divulgazione. Accettandolo, riconosci che tutti i contenuti dell'esame, comprese domande, opzioni di risposta e scenari, sono informazioni riservate e proprietà esclusiva di Anthropic e accetti di non divulgarli, riprodurli o distribuirli. Se non accetti l'accordo, la sessione termina e non viene emesso alcun rimborso.

15. Mantenimento della credenziale e ricertificazione

La credenziale Claude Certified Architect – Foundations è valida per 12 mesi dalla data di assegnazione. Poiché la tecnologia sottostante evolve rapidamente, la credenziale ha una durata

limitata affinché i titolari mantengano aggiornate le proprie conoscenze.

Per rinnovarla entro i termini, verifica le modifiche intervenute dalla certificazione e completa una valutazione gratuita, non sorvegliata, su Anthropic Partner Academy. Il rinnovo puntuale non prevede costi. Se la credenziale scade, per riottenere lo stato di certificato devi ripetere l'esame completo pagando la quota intera.

Se i contenuti dell'esame cambiano in modo significativo, Anthropic può richiedere ai titolari di ripetere l'esame completo per ricertificarsi, anziché completare la valutazione di rinnovo. I titolari restano soggetti alle regole di condotta descritte nella sezione 13.

16. Assistenza ai candidati, ricorsi e privacy

Assistenza

Per correggere il nome nella registrazione, contatta certifications-support@anthropic.com. Per tutte le altre richieste, comprese registrazione, prenotazione, misure di accomodamento e risultati, contatta l'assistenza Pearson VUE all'indirizzo pearsonvue.com/us/en/anthropic.html.

Ricorsi e reclami

Puoi presentare ricorso contro una decisione entro 14 giorni dalla data in cui ne ricevi comunicazione oppure, se il ricorso riguarda il risultato dell'esame, entro 14 giorni dalla data dell'esame. Invia il ricorso all'assistenza Pearson VUE tramite il link indicato sopra. I ricorsi vengono esaminati in base alla politica sui ricorsi del programma. L'esito dello studio che ha stabilito la soglia e i contenuti delle singole domande non possono essere oggetto di ricorso.

Privacy

I dati personali raccolti durante la registrazione e l'esame sono trattati secondo le informative sulla privacy di Anthropic e Pearson VUE e nel rispetto delle leggi applicabili sulla protezione dei dati.

17. Appendice

Tecnologie e concetti

I seguenti argomenti e tecnologie potrebbero essere inclusi nell'esame:

- Claude Agent SDK: definizioni degli agenti, loop agentici, gestione di stop_reason, hook (PostToolUse, intercettazione delle chiamate agli strumenti), creazione di sottoagenti tramite Task, configurazione di allowedTools;
- Model Context Protocol (MCP): server, strumenti e risorse MCP, flag isError, descrizioni e distribuzione degli strumenti, configurazione .mcp.json, espansione delle variabili d'ambiente;
- Claude Code: gerarchia di configurazione di CLAUDE.md (utente, progetto, directory), .claude/rules/ con ambiti di percorso nel frontmatter YAML, .claude/commands/ per i comandi slash, .claude/skills/ e frontmatter di SKILL.md (context: fork, allowed-tools, argument-hint), modalità Plan, esecuzione diretta, comando /memory, /compact, --resume, fork_session, sottoagente Explore;
- CLI di Claude Code: flag -p / --print per la modalità non interattiva, --output-format json e --json-schema per output strutturati in CI;

- Claude API: tool_use con schemi JSON, opzioni tool_choice (auto, any, selezione forzata), valori di stop_reason (tool_use, end_turn), max_tokens, prompt di sistema;
- Message Batches API: risparmio del 50% sui costi, elaborazione fino a 24 ore, custom_id per associare richieste e risposte, verifica dello stato di completamento, assenza del supporto alle chiamate agli strumenti multi-turno;
- JSON Schema: campi obbligatori e facoltativi, tipi enum, campi nullable, pattern "other" più dettagli, modalità rigorosa per eliminare gli errori di sintassi;
- Pydantic: convalida degli schemi, errori di convalida semantica, cicli di convalida e ripetizione;
- Strumenti integrati: Read, Write, Edit, Bash, Grep, Glob, relativi scopi e criteri di selezione;
- Prompting few-shot: esempi mirati per scenari ambigui, dimostrazione del formato e generalizzazione a pattern nuovi;
- Prompt chaining: scomposizione sequenziale delle attività in passaggi mirati;
- Gestione della finestra di contesto: budget di token, sintesi progressiva, effetto «lost in the middle», estrazione del contesto, file scratchpad;
- Gestione delle sessioni: ripresa delle sessioni, fork_session, sessioni nominate, isolamento del contesto;
- Punteggi di affidabilità: affidabilità a livello di campo, calibrazione con insiemi di convalida etichettati, campionamento stratificato per misurare il tasso di errore.

Argomenti inclusi

I seguenti argomenti sono esplicitamente valutati nell'esame:

- implementazione dei loop agentici: flusso di controllo basato su stop_reason, gestione dei risultati degli strumenti, condizioni di terminazione;
- orchestrazione multi-agente: pattern coordinatore-sottoagente, scomposizione delle attività, esecuzione parallela dei sottoagenti, cicli di perfezionamento iterativo;
- gestione del contesto dei sottoagenti: passaggio esplicito del contesto, persistenza strutturata dello stato, ripristino dopo un arresto anomalo tramite manifest;
- progettazione delle interfacce degli strumenti: descrizioni efficaci, scelta tra suddividere o consolidare gli strumenti, denominazione per ridurre le ambiguità;
- progettazione di strumenti e risorse MCP: risorse per cataloghi di contenuti, strumenti per azioni, qualità delle descrizioni per favorirne l'adozione;
- configurazione dei server MCP: ambito di progetto o utente, espansione delle variabili d'ambiente, accesso simultaneo a più server;
- gestione e propagazione degli errori: risposte strutturate, errori temporanei rispetto a errori aziendali o di autorizzazione, recupero locale prima dell'escalation;
- decisioni di escalation: criteri espliciti, rispetto delle preferenze dei clienti, individuazione delle lacune nelle regole;
- configurazione di CLAUDE.md: gerarchia (utente, progetto, directory), pattern @import, .claude/rules/ con pattern glob;
- comandi personalizzati e skill: ambito di progetto o utente, context: fork, allowed-tools, frontmatter argument-hint;
- modalità Plan ed esecuzione diretta: valutazione della complessità, decisioni architetturali, modifiche a un singolo file;
- perfezionamento iterativo: esempi di input e output, iterazione guidata dai test, pattern di intervista, risoluzione sequenziale o parallela dei problemi;
- output strutturato tramite tool_use: progettazione degli schemi, configurazione tool_choice, campi

nullable per evitare allucinazioni;
- prompting few-shot: esempi mirati a scenari ambigui, coerenza del formato, riduzione dei falsi positivi;
- elaborazione batch: idoneità della Message Batches API, valutazione della tolleranza alla latenza, gestione degli errori tramite custom_id;
- ottimizzazione della finestra di contesto: riduzione degli output verbosi degli strumenti, estrazione di fatti strutturati, ordinamento degli input consapevole degli effetti della posizione;
- flussi di revisione umana: calibrazione dell'affidabilità, campionamento stratificato, segmentazione dell'accuratezza per tipo di documento e campo;
- provenienza delle informazioni: corrispondenza tra affermazioni e fonti, gestione dei dati temporali, annotazione dei conflitti, segnalazione delle lacune di copertura.

Argomenti esclusi

I seguenti argomenti correlati non saranno inclusi nell'esame:

- fine-tuning dei modelli Claude o addestramento di modelli personalizzati;
- autenticazione, fatturazione o gestione dell'account Claude API;
- implementazione dettagliata di specifici linguaggi o framework di programmazione, oltre quanto necessario per configurare strumenti e schemi;
- distribuzione o hosting di server MCP (infrastruttura, rete, orchestrazione di container);
- architettura interna, processo di addestramento o pesi dei modelli Claude;
- Constitutional AI, RLHF o metodologie di addestramento per la sicurezza;
- modelli di embedding o dettagli implementativi dei database vettoriali;
- uso del computer (automazione del browser o interazione con il desktop);
- analisi di immagini e capacità visive;
- implementazione di API in streaming o di eventi server-sent;
- limiti di frequenza, quote o calcoli dei prezzi API;
- OAuth, rotazione delle chiavi API o dettagli dei protocolli di autenticazione;
- configurazioni di specifici provider cloud (AWS, GCP, Azure);
- benchmark delle prestazioni o metriche di confronto tra modelli;
- dettagli implementativi della memorizzazione nella cache dei prompt, oltre alla conoscenza della sua esistenza;
- algoritmi di conteggio dei token o dettagli sulla tokenizzazione.

18. Controllo del documento

1.0	**Aggiornamenti di formattazione e impaginazione**	**Luglio 2026**
0.2	Revisione della bozza	Giugno 2026
0.1	Bozza iniziale	Febbraio 2026